

YOLO11n Instance Segmentation App

How the FastAPI backend and Streamlit frontend work together, and how to share and deploy the app

01 What the app does

You open a web page, upload a photo and click a button. The app finds every object it recognises, such as people, buses and dogs. It paints each object with its own colour and draws a labelled box around it. You also get a table listing each object, how confident the model is and how many pixels it covers.

Under the hood there are two separate programs. **Streamlit** draws the web page. **FastAPI** runs the YOLO11n segmentation model. They talk to each other over HTTP, exactly like a browser talks to a website.

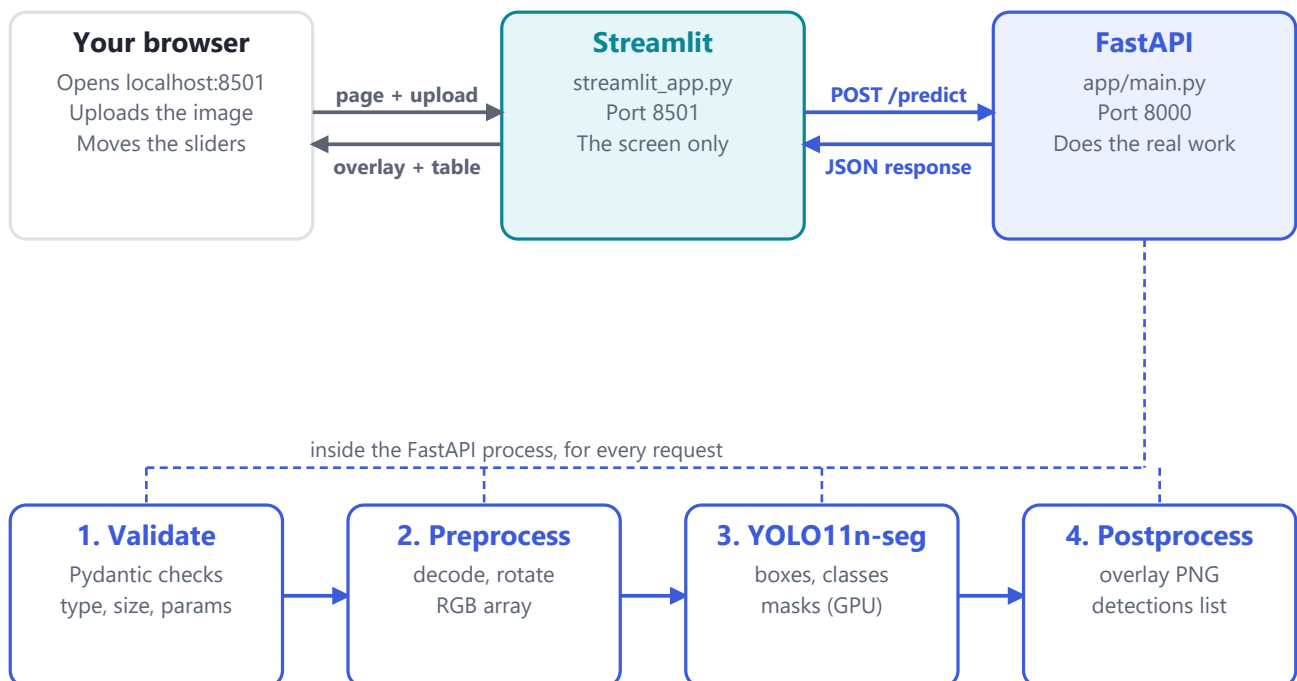


Figure 1. Streamlit is the screen. FastAPI does the work. The only link between them is an HTTP request.

02 Project structure

File	Role
<code>app/main.py</code>	The FastAPI app. Loads the model once at startup and defines GET /health and POST /predict .
<code>app/preprocess.py</code>	Checks the upload, decodes the bytes into an image, fixes phone-camera rotation and converts it to RGB.
<code>app/postprocess.py</code>	Turns YOLO's raw output into a coloured overlay and a clean list of detections, then encodes the overlay as PNG.
<code>app/schemas.py</code>	All Pydantic models: the input rules, the parameter limits and the shape of the response.
<code>streamlit_app.py</code>	The user interface. Sends the image to the API and shows what comes back.
<code>requirements.txt</code>	The Python packages the project needs.

03 Step by step: what happens when you click the button

Follow one image from your browser to the model and back again.

1 Startup, once

When you run `uvicorn`, FastAPI loads the YOLO11n weights into memory. It runs one dummy prediction to warm up the GPU, so the first real request is fast. This happens once, not on every request.

2 You upload an image

You open Streamlit on port 8501 and pick an image. Streamlit keeps the file's bytes in memory. The sidebar sliders hold your settings for confidence, IoU threshold, image size and mask opacity.

3 Streamlit sends the request

When you click **Run segmentation**, Streamlit uses the [requests](#) library to POST to <http://localhost:8000/predict>. The image travels as a multipart file and the slider values travel as form fields.

4 FastAPI checks the parameters

Before any of your code runs, FastAPI reads the form fields and passes them through the [PredictParams](#) Pydantic model. A bad value such as `conf=1.5` is rejected with a 422 error right here.

5 Preprocessing checks the file

In `preprocess.py` the file goes through three gates:

- The [UploadedImage](#) model checks the content type and file size.
- Pillow decodes the bytes. This catches fake or truncated files.
- The [DecodedImage](#) model checks the real format and the width and height.

The image is then rotated upright and converted into an RGB array of pixels.

6 Inference

The array goes to YOLO in a background thread, so the server stays responsive while the GPU works. YOLO returns boxes, class labels, confidence scores and a mask for each object.

7 Postprocessing

In `postprocess.py` each mask is scaled up to the original image size and blended over the photo in its own colour. Boxes and labels are drawn on top. The overlay is saved as PNG and then encoded as a base64 string, which lets an image travel inside JSON.

8 The response goes back

FastAPI checks the result against the `PredictResponse` model and sends the JSON back to Streamlit.

9 Streamlit shows the result

Streamlit decodes the base64 string back into an image and shows it beside the original. It fills the detections table and offers a download button.

Key idea. Streamlit never touches the model. It only sends an image and displays what comes back. If you replaced Streamlit with a phone app, FastAPI would not need to change at all.

04 Input validation

Every bad input gets a clear error instead of a crash. These rules were tested against the running API.

Problem	Status	Where it is caught
File is not JPEG, PNG, BMP or WEBP	422	UploadedImage model
Empty file, or larger than 10 MB	422	UploadedImage model
Bytes are not a real image	400	Pillow decode in preprocess.py
Image is truncated or corrupt	400	Pillow verify in preprocess.py
Width or height outside 32 to 8192 px	422	DecodedImage model
<code>conf</code> , <code>iou</code> or <code>alpha</code> outside 0 to 1	422	PredictParams model
<code>imgsz</code> not a multiple of 32, or outside 32 to 1920	422	PredictParams model

A 422 means the request broke a rule. A 400 means the file itself is unusable.

05 What you get back

This is the real overlay the API returned for a test photo. Each object has its own colour, so two people standing side by side are easy to tell apart.

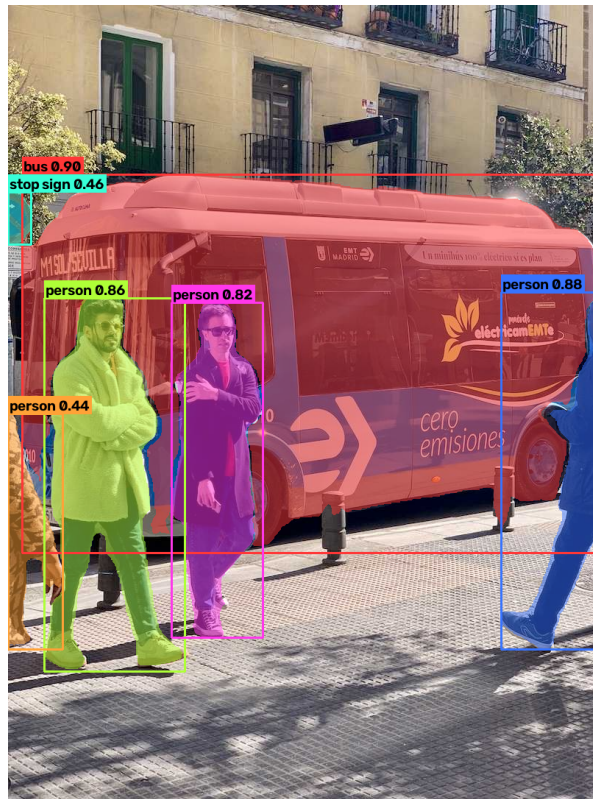


Figure 2. Six detections found in about 0.13 seconds on a GTX 1650 Ti.

The JSON response looks like this, shortened:

```
{
  "filename": "bus.jpg",
  "width": 810,
  "height": 1080,
  "num_detections": 6,
  "detections": [
    {
      "class_name": "bus",
      "confidence": 0.8985,
      "box": {"x1": 21.3, "y1": 229.9, "x2": 801.1, "y2": 744.2},
      "mask_area_px": 261956
    }
  ],
  "inference_ms": 125.6,
  "overlay_png_base64": "iVBORw0KGgo..."
}
```

06 Running it on your machine

Open two terminals in the project folder. Start the API in the first one:

```
conda activate myenv2
uvicorn app.main:app --port 8000
```

Start the user interface in the second one:

```
conda activate myenv2
streamlit run streamlit_app.py
```

Then open <http://localhost:8501> for the app, or <http://localhost:8000/docs> for FastAPI's interactive API page. You can also test the API without Streamlit:

```
curl -X POST http://localhost:8000/predict -F "file=@bus.jpg;type=image/jpeg" -F conf=0.3
```

07 Could you have skipped FastAPI?

Yes. Streamlit runs Python, so it could load YOLO itself and call the preprocess and postprocess functions directly. For a personal demo that version is simpler. You get one process, no network hop and no base64 encoding.

FastAPI earns its place in these situations:

Reason	Why it matters
Other clients	A phone app, another service or a script can call /predict . A Streamlit-only app can only be used through its own page.
Separate scaling	The model can run on a GPU machine while the UI runs somewhere cheap. You can add more API workers when traffic grows.
One loaded model	Streamlit reruns the whole script on every click. Without careful caching it would reload the model each time.
A clear contract	The Pydantic schemas define exactly what goes in and comes out. FastAPI builds the /docs page from them for free.
Swappable frontend	You could replace Streamlit with React later without touching any model code.

Tip. For learning or a quick demo, Streamlit alone is fine. If anyone or anything other than you will use the model, keep the split. It is the standard way to serve ML models.

08 Opening the app from another computer

Only Streamlit needs to be reachable. The call to FastAPI happens inside the Streamlit process on your machine, not in the visitor's browser. So FastAPI can stay on localhost, and only port 8501 has to be opened.

Option A: same Wi-Fi or office network

No public link is needed.

1 Find your local IP address

Run `ipconfig` and note the IPv4 address of your Wi-Fi adapter, for example `192.168.1.42`. Streamlit also prints it as the Network URL when it starts.

2 Allow port 8501 through Windows Firewall

Run this once in an administrator PowerShell. Also make sure your Wi-Fi is set to **Private** in Windows settings.

```
New-NetFirewallRule -DisplayName "Streamlit 8501" -Direction Inbound `
  -Protocol TCP -LocalPort 8501 -Action Allow
```

3 Open it on the other computer

Browse to `http://192.168.1.42:8501`, using your own IP. If it does not load, start Streamlit with `--server.address 0.0.0.0`.

Option B: a different network, using a tunnel

A tunnel gives your local port a temporary public HTTPS link. Both of these tools have free tiers:

```
cloudflared tunnel --url http://localhost:8501
ngrok http 8501
```

Each prints a link such as `https://random-name.trycloudflare.com` that anyone can open. Your computer must stay on with both servers running.

Watch out. The app has no login. Anyone with the link can upload images and use your GPU. Share tunnel links privately, and consider ngrok's `--basic-auth` option to add a password.

09 Deploying with Docker on AWS EC2

For a permanent link, package the app with Docker and run it on a cloud machine. The machine's public IP becomes your link, for example `http://3.110.45.12:8501`.

1 Two containers

One runs FastAPI and one runs Streamlit. A `docker-compose.yml` file starts both on a shared private network.

2 Containers talk by name

Streamlit reaches the API at <http://api:8000> through the `API_URL` variable the app already reads. Port 8000 never needs to be public.

3 Launch an EC2 instance

Install Docker, clone the project from GitHub and run `docker compose up -d`.

4 Open one port

In the instance's security group, allow inbound TCP on 8501. Limit SSH on port 22 to your own IP.

5 Keep the link stable

The public IP changes when an instance stops and starts. An Elastic IP keeps it fixed.

Choices to make

Decision	Recommendation
CPU or GPU	YOLO11n is small, so a CPU instance like t3.medium works, at a few hundred milliseconds per image. A GPU instance like g4dn.xlarge is much faster but costs several times more per hour.
Docker image size	Install the CPU-only build of torch on a CPU instance. The CUDA build adds several gigabytes for nothing. Copy yolo11n-seg.pt into the image so it is not downloaded on every start.
HTTPS and a domain	Point a domain at the Elastic IP and put Caddy or nginx in front to handle the certificate.
Cost	EC2 bills by the hour while the instance runs, even when idle. A t3.medium left on for a month is roughly 30 US dollars.
Other hosts	DigitalOcean, Google Cloud and Azure work the same way. Render, Railway and Cloud Run take a Docker image and give you an HTTPS link directly.

10 Git quick reference

Command	What it does
<code>git switch -c my-branch</code>	Create a new branch and switch to it.
<code>git checkout -b my-branch</code>	The older command that does the same thing.
<code>git branch my-branch</code>	Create a branch without switching to it.
<code>git push -u origin my-branch</code>	Push a new branch to GitHub and link it, so later pushes need no arguments.

Uncommitted changes move with you when you create and switch to a new branch.